

ParcheggIA - Guida completa per l'esame

Ultimo aggiornamento: 15 luglio 2026

Scopo del documento: avere una guida tecnica completa per rispondere a domande sull'implementazione, sulle scelte architetturali e sui flussi interni del progetto.

Questo documento non e' pensato come README per l'avvio rapido, ma come materiale di ripasso per l'orale. Contiene anche dettagli piccoli che possono essere utili se viene chiesto "come funziona esattamente?".

1. Idea del progetto

ParcheggIA e' una demo cloud-native che stima la probabilita' di trovare parcheggio nelle immediate vicinanze dell'utente a Catania.

L'utente vede una interfaccia simile a un navigatore:

- mappa scura stile Android Auto / Google Maps;
- posizione utente;
- strade colorate con heatmap continua;
- marker sui segmenti con percentuale stimata;
- marker dei parcheggi vicini;
- suggerimento AI;
- Text-to-Speech;
- simulazione di guida;
- fallback senza API key.

Il problema non e' solo mostrare una mappa. Il progetto dimostra una architettura completa:

- frontend web;
- microservizi backend;
- database geospaziale;
- cache;
- broker eventi;
- AI;
- TTS;
- Docker;
- Kubernetes locale;
- AWS reale;
- Terraform;
- GitHub Actions.

La domanda a cui il sistema cerca di rispondere e':

"Sono in questa posizione: quale tratto di strada vicino a me conviene provare per cercare parcheggio?"

2. Stato reale rispetto ai requisiti preliminari

Requisito iniziale	Stato reale	Note
React + TypeScript	Non implementato	Sostituito da HTML/CSS/JavaScript vanilla per ridurre complessita'.
Android Studio / Kotlin	Non implementato	L'esperienza Android Auto e' simulata da web app responsive.
MapLibre / Leaflet	Implementato con MapLibre	Leaflet e' stato abbandonato per usare WebGL, pitch, bearing e layer vettoriali.

Requisito iniziale	Stato reale	Note
Spring Boot	Non implementato	Sostituito da Nginx gateway + microservizi FastAPI.
FastAPI	Implementato	Tutti i microservizi backend applicativi sono FastAPI.
PostgreSQL + PostGIS	Implementato	Database principale geospaziale.
Redis	Implementato	Cache, sessioni, segnali dinamici, budget API.
RabbitMQ	Implementato	Eventi asincroni e scenari demo.
MongoDB	Non implementato	Valutato opzionale; PostGIS + Redis + RabbitMQ bastano per la demo.
NVIDIA Nemotron	Implementato opzionale	Suggerimenti AI live se key presente, fallback simulato se assente.
Docker	Implementato	Ogni servizio gira in container.
Docker Compose	Implementato	Avvio locale con docker compose up -d --build .
Kubernetes	Implementato	Manifest per demo locale e cloud.
Multipass/k3s	Parziale	Playbook presenti, flusso validato con k3d/k3s.
AWS	Implementato	EKS, RDS, ElastiCache, Amazon MQ, ECR, SSM, Lambda auto-down.
Terraform	Implementato	Provisioning AWS.
Ansible	Parziale	Playbook presenti per setup locale/k3d/Multipass; non e' il flusso principale.
GitHub Actions	Implementato	CI, build, ECR, Terraform, deploy, cloud-down.

Da dire all'esame: alcune tecnologie iniziali sono state sostituite con scelte piu' pragmatiche. Non e' una mancanza casuale: e' una decisione progettuale motivabile.

3. Stack tecnologico finale

Frontend

- HTML statico.
- CSS custom.
- JavaScript vanilla.
- MapLibre GL JS.
- PMTiles per basemap vettoriale locale di Catania.
- Material Symbols per icone.
- Browser Geolocation API.
- Web Speech API come fallback TTS.

File principali:

- `frontend/index.html`
- `frontend/styles.css`
- `frontend/app.js`
- `frontend/assets/catania.pmtiles`
- `frontend/nginx.conf`

Backend

Tutti i servizi applicativi sono Python FastAPI:

- `zone-service`
- `prediction-service`
- `location-service`
- `ingestion-service`
- `nemotron-service`
- `admin-service`

Il gateway e' Nginx:

- `api-gateway`

Persistenza e messaging

- PostgreSQL 16 + PostGIS 3.4
- Redis 7
- RabbitMQ 3 Management

AI e provider esterni

- TomTom Traffic Flow API
- TomTom Traffic Incidents API
- TomTom Search API
- NVIDIA Nemotron, modello predefinito `nvidia/nemotron-3-nano-30b-a3b`
- ElevenLabs TTS, modello `eleven_flash_v2_5`

Tutti opzionali. Senza chiavi il progetto funziona comunque con fallback.

Cloud

- AWS EKS
- AWS RDS PostgreSQL
- AWS ElastiCache Redis
- Amazon MQ for RabbitMQ
- AWS ECR
- AWS SSM Parameter Store
- CloudWatch Logs
- Lambda + EventBridge Scheduler per auto-spegnimento
- Terraform
- Kubernetes
- GitHub Actions

4. Come si avvia il progetto

Locale con Docker Compose

Comando:

```
docker compose up -d --build
```

URL:

`http://localhost:8080`

API Gateway:

`http://localhost:8000`

Verifica:

```
docker compose ps
scripts/smoke_test.sh
scripts/e2e_demo_test.sh
```

Spegnimento:

```
docker compose down
```

Reset completo database:

```
docker compose down -v
```

Simulazione cloud locale

Comando:

```
scripts/cloud_sim_local_up.sh
```

URL:

```
http://localhost:18080
```

Cosa simula:

- k3d/k3s al posto di EKS;
- pod PostgreSQL/PostGIS al posto di RDS;
- pod Redis al posto di ElastiCache;
- pod RabbitMQ al posto di Amazon MQ;
- immagini Docker locali importate nel cluster al posto di ECR;
- port-forward locale al posto del Load Balancer AWS.

Stop:

```
scripts/cloud_sim_local_down.sh
```

Cloud AWS reale

Comando principale:

```
CONFIRM_APPLY=apply-parcheggia-dev scripts/cloud_demo_up.sh
```

Spegnimento:

```
CONFIRM_DESTROY=destroy-parcheggia-dev scripts/cloud_down.sh
```

Regola importante:

- AWS crea risorse a costo continuo.
- Il progetto ha auto-spegnimento via Lambda/EventBridge/GitHub Actions.
- Comunque dopo la demo bisogna lanciare sempre `cloud_down.sh`.

5. Servizi Docker Compose

postgres

Immagine:

```
postgis/postgis:16-3.4
```

Ruolo:

- database principale;
- contiene dati geografici;
- abilita query spaziali tramite PostGIS;
- conserva segmenti OSM, zone legacy, parcheggi, report, road network.

Porta locale:

5432

Credenziali locali:

database: parcheggio
utente: parcheggio
password: parcheggio

db-init

Ruolo:

- container one-shot;
- attende PostgreSQL;
- esegue migrazione `001_create_zones.sql`;
- importa `data/osm/catania_segments.sql` se i segmenti OSM non sono presenti;
- applica `data/osm/catania_blue_overrides.sql`.

Questo e' importante per l'esame: chi clona la repo non deve importare manualmente i dati. Il database si inizializza automaticamente.

redis

Immagine:

redis:7

Porta:

6379

Ruolo:

- sessioni live;
- cache prediction;
- segnali dinamici;
- budget TomTom;
- cache TomTom;
- cache AI/TTS lato servizio.

rabbitmq

Immagine:

rabbitmq:3-management

Porte:

5672 AMQP

15672 management UI

Credenziali:

parcheggio / parcheggio

Ruolo:

- eventi asincroni;
- scenari demo;
- report utente;
- traffico/eventi cittadini;
- aggiornamenti posizione.

zone-service

Porta:

8001

Dipendenze:

- PostgreSQL/PostGIS;
- Redis;
- RabbitMQ.

Ruoli:

- serve segmenti stradali;
- calcola segmento corrente;
- restituisce segmenti vicini;
- espone road network;
- gestisce report segment-level;
- consumer RabbitMQ;
- supporta ancora API legacy `/zones`.

ingestion-service

Porta:

8002

Ruoli:

- chiama TomTom quando configurato;
- pubblica eventi su RabbitMQ;
- gestisce scenari demo;
- gestisce budget guard;
- espone endpoint per POI parcheggi;
- cache TomTom in Redis.

nemotron-service

Porta:

8003

Ruoli:

- genera suggerimenti AI;
- usa Nemotron se key presente;
- usa fallback simulato se key assente;
- genera TTS con ElevenLabs se key presente;
- usa fallback browser lato frontend se ElevenLabs manca;
- cache dei suggerimenti e audio.

prediction-service

Porta:

8004

Ruoli:

- calcola parkability percent;
- genera status visuale;
- combina baseline, report, segnali Redis, ora/giorno;
- espone `segment-heatmap`;
- mantiene compatibilit  legacy `/heatmap`.

location-service

Porta:

8005

Ruoli:

- crea live session;
- aggiorna posizione utente;
- salva stato sessione in Redis;
- chiama zone-service e prediction-service;
- pubblica evento `user.location.updated` su RabbitMQ.

admin-service

Porta:

8006

Ruoli:

- stato sorgenti;
- eventi recenti;
- reset demo;
- diagnostica Redis/RabbitMQ/Zone.

api-gateway

Porta:

8000

Tecnologia:

- Nginx.

Ruolo:

- unico punto di ingresso API;
- instrada `/api/v1/live-sessions` al location-service;
- instrada `/api/v1/segment-heatmap` al prediction-service;
- instrada `/api/v1/tomtom` all'ingestion-service;
- instrada `/ai` al nemotron-service;
- instrada il resto delle API al zone-service;
- gestisce CORS.

frontend

Porta:

8080

Ruolo:

- serve la GUI.

6. Architettura logica

Schema mentale:

```
Browser / MapLibre
|
v
API Gateway Nginx
```

```

|
+---> Zone Service -----> PostgreSQL/PostGIS
|           |
|           +-----> Redis
|           +-----> RabbitMQ consumer
|
+---> Prediction Service ---> PostgreSQL/PostGIS
|           -> Redis
|
+---> Location Service -----> Redis
|           -> RabbitMQ
|           -> Zone + Prediction
|
+---> Ingestion Service -----> TomTom opzionale
|           -> Redis budget/cache
|           -> RabbitMQ
|
+---> Nemotron Service -----> Nemotron opzionale
|           -> ElevenLabs opzionale
|
+---> Admin Service -----> Redis/RabbitMQ/Zone

```

Principio fondamentale:

Il frontend non chiama mai direttamente TomTom, Nemotron o ElevenLabs.

Motivi:

- non esporre API key;
- controllare budget;
- centralizzare cache;
- avere fallback;
- poter sostituire provider senza cambiare UI.

7. Frontend in dettaglio

File principale:

frontend/app.js

Costanti importanti:

```

LOCAL_RADIUS_M = 500
PARKING_POI_RADIUS_M = 500
TOMTOM_PREDICTION_REFRESH_MS = 5 minuti
TOMTOM_PREDICTION_SETTLE_MS = 700 ms
ROAD_NETWORK_RADIUS_M = 700
ROAD_SNAP_RADIUS_M = 85
MAP_DEFAULT_ZOOM = 18.6
MAP_TRACKING_ZOOM = 19.1
TOMTOM_POI_TEST_CELLS = 3

```

Perche' MapLibre

MapLibre e' stato scelto perche':

- supporta WebGL;
- supporta pitch e bearing;
- gestisce layer vettoriali;
- gestisce heatmap native;
- gestisce marker DOM;

- rende possibile un'interfaccia da navigatore.

Leaflet sarebbe stato piu' semplice ma meno adatto per:

- camera inclinata;
- linee continue ad alte prestazioni;
- heatmap e marker sovrapposti;
- stile Android Auto.

Basemap

La basemap e' locale:

`frontend/assets/catania.pmtiles`

Vantaggi:

- niente API key MapTiler/Stadia/TomTom Map Display;
- niente consumo tile esterne;
- funziona anche offline/localmente;
- coerente con l'idea di demo riproducibile.

Heatmap

La heatmap e' basata su segmenti, non su grandi zone.

Per renderla continua:

- il segmento LineString viene campionato lungo la strada;
- non viene usato solo il punto medio;
- si usano layer MapLibre con effetto morbido;
- ribbon lineari danno continuita' lungo la via;
- marker separati mostrano percentuale e tipo sosta.

Problema risolto:

- prima si vedevano "pallini";
- poi sono stati rimossi core puntuali troppo netti;
- il risultato attuale segue la geometria della strada.

Marker segmenti

I marker mostrano:

- percentuale al centro;
- bordo colorato in base alla tipologia o stato;
- anello verde rotante sui target suggeriti dall'AI.

Sono pensati per essere leggibili senza coprire troppo la mappa.

Marker parcheggi

I parcheggi possono arrivare da:

- TomTom Search API se configurata;
- fallback simulato se senza API key.

Il frontend li mostra come marker animati. Quando entrano nel campo visivo:

- animazione di comparsa;
- spin breve;
- effetto tipo punto di interesse.

Movimento utente

Ci sono varie modalita':

- posizione reale browser se autorizzata;
- click-to-drive su strada;
- frecce tastiera come fallback;
- simulazione automatica da 500 metri.

Inizialmente le frecce erano poco naturali. La soluzione piu' stabile per test PC e' stata:

- click sulla strada;
- snap alla road network OSM;
- simulazione guida hardcoded/dimostrativa.

Snap alla strada

Lo snap non usa direttamente i segmenti parcheggio.

Motivo:

- i segmenti parcheggio possono essere spezzati e non sempre formano una rete navigabile;
- rischiavano di bloccare l'utente su tratti non collegati.

Soluzione:

- road network separata con `road_edges` e `road_nodes`;
- endpoint `/api/v1/road-network`;
- snap al bordo stradale piu' vicino entro soglia.

Simulazione guida

La simulazione serve a mostrare:

- spostamento utente;
- camera che segue il senso di marcia;
- heatmap dinamica;
- suggerimenti AI;
- marker parcheggi;
- TTS.

Non e' routing reale.

Da dire chiaramente:

La simulazione e' dimostrativa; il routing reale e' uno sviluppo futuro con OSRM, Valhalla o TomTom Routing.

Tema chiaro/scuro

La GUI supporta:

- tema scuro;
- tema chiaro;
- toggle persistito in localStorage.

Meteo

Il progetto ha introdotto dati meteo real time lato frontend, dove possibile.

Audio/TTS

Il tasto volume:

- muta/smuta il TTS;
- se mutato cancella eventuale parlato in corso;

- non controlla l'audio di sistema.
-

8. Modello dati

Tabella legacy zones

Esiste ancora per compatibilit  e demo iniziale.

Campi principali:

- id
- name
- city
- zone_type
- baseline_capacity_estimate
- polygon
- created_at

La geometria  :

GEOMETRY(POLYGON, 4326)

Indice:

GIST su polygon

Tabella parking_lots

Inizialmente collegata alle zone.

Campi principali:

- id
- name
- operator
- zone_id
- location
- total_capacity
- pricing_info
- is_park_and_ride

La geometria  :

GEOMETRY(POINT, 4326)

Tabella user_reports

Report utente legacy/compatibili:

- found_spot
- full_zone
- released_spot
- parking_closed

Tabelle moderne segment-level

Il modello moderno   basato su:

- parking_segments;
- road_edges;
- road_nodes;
- report segment-level;
- segnali Redis.

`parking_segments` rappresenta piccoli tratti stradali.

Campi logici:

- `id`
- `street_name`
- `geometry` LINESTRING
- `length_m`
- `parking_type`
- `tariff_zone`
- `price_label`
- `time_rules`
- `source`
- `source_confidence`

Tipi sosta:

- `blue`: strisce blu;
- `probable_free`: probabilmente libero;
- `restricted`: limitato;
- `unknown`: ignoto.

Nota importante:

“Probabile libero” non e’ un’informazione legale. Significa che il sistema non ha trovato override blu/restricted e inferisce disponibilita’ probabile, ma l’utente deve verificare segnaletica.

Road network

La road network e’ separata dai segmenti parcheggio.

Motivo:

- i segmenti parcheggio servono alla stima;
- la rete stradale serve alla navigazione/snap;
- mescolare le due cose creava problemi di navigabilita’.

9. Import OSM

File importanti:

```
data/osm/catania-overpass.json
data/osm/catania_segments.sql
data/osm/catania_blue_overrides.sql
scripts/import_osm_segments.py
scripts/check_osm_import.py
scripts/check_road_backed_segments.py
```

Cosa significa importare OSM

OpenStreetMap contiene geometrie stradali. Il progetto usa questi dati per:

- ottenere le vie reali;
- spezzare le vie in segmenti;
- assegnare nomi strada;
- creare rete stradale;
- sovrapporre heatmap a strade reali.

La pipeline produce un SQL importabile:

```
data/osm/catania_segments.sql
```

Perche' non ricavare i segmenti da MapLibre

MapLibre renderizza tile vettoriali ma non e' un database stradale interrogabile in modo affidabile per segmentazione applicativa.

Problemi:

- i tile sono ottimizzati per rendering, non per analisi;
- possono essere semplificati in base allo zoom;
- non sono pensati per mantenere ID persistenti;
- non bastano per report, prediction e storico.

Per questo i segmenti sono importati in PostGIS.

Override strisce blu

Gli override servono per correggere dati OSM incompleti.

Fonti:

- Comune/AMTS dove disponibili;
- CSV manuale;
- SQL di override.

Effetto:

- tratti noti diventano **blue**;
- possono ricevere tariffa/orario;
- possono aumentare confidence.

10. Prediction model

Il modello attuale e' rule-based, non machine learning supervisionato.

Motivo:

- non c'e' ancora storico reale di occupazione;
- serve una demo spiegabile;
- deve funzionare anche senza provider esterni;
- e' piu' facile da testare.

Baseline

Valori concettuali:

Tipo sosta	Baseline
strisce blu	circa 42%
probabile libero	circa 48%
limitato	circa 14%
ignoto	circa 42%

Status visuale

Funzione `status_from_percent`:

```
< 20%    very_difficult
20-39%    difficult
40-59%    uncertain
60-79%    good
>= 80%    favorable
```

Effetto report

Funzione `report_adjustment`:

- `found_spot`: aumenta;
- `released_spot`: aumenta;
- `full_zone`: diminuisce;
- `parking_closed`: diminuisce molto.

Il delta e' limitato tra:

-15 e +15

Formula:

`found * 5 + released * 6 - full * 7 - closed * 8`

con clamp finale.

Effetto segnali Redis/TomTom

Funzione `signal_delta`:

Input:

- `traffic_pressure`
- `parking_lot_availability`
- `event_pressure`

Effetto:

- traffico alto peggiora;
- eventi/incidenti peggiorano;
- disponibilita' parcheggi migliora o peggiora;
- delta limitato tra -35 e +20.

Formula concettuale:

`(parking_availability - 0.5) * 35`
`- traffic_pressure * 22`
`- event_pressure * 18`

Fascia oraria

Il modello considera anche ora/giorno:

- ore trafficate peggiorano;
- notte migliora;
- weekend puo' cambiare la stima.

Confidence

La confidence non e' la percentuale di parcheggio.

Differenza:

- `parkability_percent`: probabilita' stimata di trovare posto;
- `confidence`: quanto il sistema e' sicuro della stima.

La confidence influenza:

- opacita';
- morbidezza visiva;
- intensita' heatmap;
- interpretazione del suggerimento.

11. TomTom

API usate

Il progetto usa:

- Traffic Flow API;
- Traffic Incidents API;
- Search API.

Non usa ancora in GUI:

- Routing API;
- Map Display tiles TomTom.

Perche' usare TomTom

TomTom non fornisce direttamente disponibilita' on-street dei parcheggi con le API configurate.

Fornisce pero':

- traffico;
- velocita' corrente;
- velocita' free-flow;
- incidenti;
- ritardi;
- POI parcheggi.

Questi segnali sono utili per modificare la stima.

Traffic Flow

Endpoint nel codice:

`https://api.tomtom.com/traffic/services/4/flowSegmentData/absolute/10/json`

Output utile:

- `currentSpeed`
- `freeFlowSpeed`
- `confidence`
- `roadClosure`

Calcolo:

`traffic_pressure = 1 - currentSpeed / freeFlowSpeed`

Se c'e' road closure:

`traffic_pressure = 1.0`

Campionamento Flow

Il progetto non chiama TomTom per ogni segmento.

Usa massimo punti campione:

- centro;
- nord;
- est;
- sud;
- ovest.

Quindi una stima live puo' consumare fino a circa:

`5 chiamate Traffic Flow + 1 chiamata Incidents`

Traffic Incidents

Endpoint:

`https://api.tomtom.com/traffic/services/5/incidentDetails`

Usa bbox intorno all'utente.

Campi utili:

- tipo incidente;
- categoria;
- ritardo;
- lunghezza;
- probabilita';
- numero report;
- from/to.

Incident pressure

Gli incidenti vengono convertiti in pressione evento:

- road closed pesa molto;
- accident pesa molto;
- jam pesa molto;
- lavori o veicolo fermo pesano meno.

L'effetto massimo viene normalizzato tra 0 e 1.

Search API parcheggi

Endpoint:

`https://api.tomtom.com/search/2/nearbySearch/.json`

Parametri:

- lat
- lon
- radius
- limit
- countrySet=IT
- language=it-IT
- categorySet

Categorie:

- 7369: open parking area;
- 7313: parking garage.

Nota: in una fase precedente era stato citato 7311; nel codice attuale e' 7313.

Pulizia indirizzi parcheggi

Il backend pulisce indirizzi TomTom per non mostrare:

- CAP;
- "Catania";
- "Italia";
- "Italy";
- sigla CT quando ridondante.

Questo e' stato fatto per evitare suggerimenti tipo:

Via X, 95100 Catania CT, Italia

che in GUI risultano troppo lunghi.

Budget guard

Variabili:

```
TOMTOM_BUDGET_FRACTION=0.05  
TOMTOM_MONTHLY_LIMIT_TRAFFIC_FLOW=200000  
TOMTOM_MONTHLY_LIMIT_TRAFFIC_INCIDENTS=2500  
TOMTOM_MONTHLY_LIMIT_SEARCH=2500  
TOMTOM_MONTHLY_LIMIT_ROUTING=20000
```

Significa:

- in test si usa massimo 5% della quota mensile;
- il budget e' contato in Redis;
- se supera cap, il backend blocca la chiamata.

Cache TomTom

Prediction:

```
TTL = 300 secondi  
cella = circa 250 m  
raggio = 500 m
```

POI parcheggi:

```
TTL = 86400 secondi  
raggio = 500 m
```

Motivo:

- traffico cambia spesso, quindi TTL 5 minuti;
- POI parcheggi cambiano raramente, quindi cache 24 ore.

Perche' raggio 500 m

All'inizio era 900 m.

Ridotto a 500 m per:

- coprire vista attuale;
- evitare dati inutilmente lontani;
- ridurre consumo TomTom;
- rendere suggerimenti piu' locali;
- migliorare performance.

12. Nemotron

Modello

Default:

```
nvidia/nemotron-3-nano-30b-a3b
```

Base URL:

```
https://integrate.api.nvidia.com/v1
```

Timeout:

```
45 secondi
```

Cache:

```
600 secondi
```

Perche' Nano

Inizialmente era stato provato un modello piu' grande.

Nano e' stato scelto per:

- latenza minore;
- costo/tempo inferiore;
- supporto italiano sufficiente;
- suggerimenti brevi e pratici.

Cosa riceve Nemotron

Non riceve dati inventati.

Riceve:

- strada corrente;
- percentuale corrente;
- status;
- tipo sosta;
- confidence;
- segmenti vicini;
- parcheggi vicini;
- target ID disponibili;
- contesto sintetico.

Cosa deve produrre

Deve produrre JSON compatto con campi come:

- `summary`;
- `action`;
- `reason`;
- `risk`;
- `caveat`;
- `target_segment_ids`;
- `target_parking_poi_ids`.

Il frontend evidenzia solo ID validi presenti nel contesto.

Regole del prompt

Il prompt e' stato raffinato per evitare:

- termini troppo tecnici;
- "stima inferita";
- spiegazioni lunghe;
- dettagli inutili sulla segnaletica;
- inglesismi;
- consigli statici sempre uguali.

Obiettivo:

consigli pratici, brevi, in italiano, basati sulle immediate vicinanze

Fallback

Se manca la key:

- il servizio resta ready;
- segnala fallback/simulated mode;
- non espone errori bloccanti;
- permette demo locale completa.

13. ElevenLabs TTS

Modello

Default:

`eleven_flash_v2_5`

Motivo:

- minore consumo crediti;
- latenza inferiore;
- qualità sufficiente per demo.

Voce

Voice ID default:

`EXAVITQu4vr4xnSDxMaL`

Nel progetto e' stata scelta una voce femminile gia' funzionante.

Parametri

```
similarity_boost = 0.50
style_exaggeration = 0.25
output_format = mp3_22050_32
timeout = 20 secondi
```

Cache TTS

Il servizio mantiene cache in memoria:

- chiave basata su voce, modello, similarity, style e testo;
- massimo 64 elementi;
- se supera, pulisce cache.

Motivo:

- evitare consumi ripetuti;
- velocizzare ripetizioni;
- ridurre crediti ElevenLabs.

Fallback TTS

Se ElevenLabs manca:

- il frontend puo' usare Web Speech API del browser;
 - la demo resta funzionante.
-

14. Redis in dettaglio

Redis e' usato per dati temporanei e veloci.

Esempi:

- sessioni live;
- cache heatmap/prediction;
- segnali `segment:signals:*`;
- budget TomTom;
- raw events;

- reset demo;
- stato sorgenti.

Perche' Redis:

- accesso veloce;
- TTL nativo;
- adatto a cache;
- adatto a stato volatile;
- facile da sostituire con ElastiCache in cloud.

Non e' usato per dati geografici persistenti. Quelli stanno in PostGIS.

15. RabbitMQ in dettaglio

RabbitMQ gestisce comunicazione asincrona.

Exchange:

`parcheggia.events`

Eventi concettuali:

- `user.location.updated`
- `user.report.created`
- `traffic.snapshot.received`
- `parkinglot.availability.updated`
- `city.event.created`

Perche' RabbitMQ:

- dimostra messaging tra microservizi;
- disaccoppia ingestion e consumer;
- consente scenari demo;
- in AWS ha equivalente gestito Amazon MQ.

Perche' non SQS/SNS:

- erano opzionali;
 - RabbitMQ era gia' presente localmente;
 - Amazon MQ permette continuita' cloud senza cambiare modello AMQP.
-

16. PostgreSQL/PostGIS in dettaglio

PostGIS e' fondamentale per:

- nearest segment;
- segmenti entro raggio;
- bbox heatmap;
- distanza tra punto utente e geometria;
- road network;
- parking lots con coordinate.

Esempi concettuali di funzioni usate:

- `ST_DWithin`
- `ST_Distance`
- `ST_Transform`
- `ST_SetSRID`
- `ST_MakePoint`
- `ST_GeomFromText`
- indici GIST

Perche' PostGIS:

- molto adatto a dati geografici;
 - query spaziali efficienti;
 - meglio di calcoli manuali in memoria;
 - mappatura diretta su RDS PostgreSQL in AWS.
-

17. API principali

Health e readiness

Ogni servizio espone:

```
/health  
/ready  
/metrics
```

Segmenti

```
GET /api/v1/segments  
GET /api/v1/segments/current?lat=&lon=  
GET /api/v1/segments/nearby?lat=&lon=&radius_m=&limit=  
GET /api/v1/segments/{segment_id}  
GET /api/v1/segments/{segment_id}/prediction  
GET /api/v1/segment-heatmap?bbox=&zoom=  
POST /api/v1/segment-reports
```

Road network

```
GET /api/v1/road-network?lat=&lon=&radius_m=
```

Live sessions

```
POST /api/v1/live-sessions/start  
POST /api/v1/live-sessions/{session_id}/location  
GET /api/v1/live-sessions/{session_id}/nearby-segments  
POST /api/v1/live-sessions/{session_id}/stop
```

TomTom

```
POST /ingestion/traffic/tomtom/publish  
GET /ingestion/traffic/tomtom/probe  
GET /ingestion/traffic/tomtom/budget  
GET /api/v1/tomtom/parking-pois
```

AI

```
GET /ai/ready  
POST /ai/explain  
POST /ai/tts
```

Admin

```
GET /api/v1/admin/source-health  
GET /api/v1/admin/events  
POST /api/v1/admin/demo-scenarios/reset
```

Legacy zones

Esistono ancora:

```
GET /api/v1/zones
GET /api/v1/zones/current
GET /api/v1/zones/nearby
GET /api/v1/heatmap
POST /api/v1/reports
```

Motivo:

- compatibilita' con smoke test e demo iniziali;
 - transizione graduale da zone a segmenti.
-

18. Flusso dati: utente si muove

Scenario:

utente clicca su una strada

Passaggi:

1. Frontend riceve click.
 2. Frontend cerca punto piu' vicino sulla road network locale.
 3. Se entro soglia, aggiorna posizione utente.
 4. Frontend chiama gateway.
 5. Gateway inoltra a location-service.
 6. Location-service salva sessione in Redis.
 7. Location-service chiede a zone-service il segmento corrente.
 8. Zone-service interroga PostGIS.
 9. Location-service chiede a prediction-service la prediction.
 10. Prediction-service legge PostGIS e Redis.
 11. Prediction-service calcola percentuale.
 12. Location-service restituisce current segment, prediction, nearby segments.
 13. Frontend aggiorna marker, chip e heatmap.
 14. Se il segmento cambia e la percentuale cambia, parte richiesta AI.
 15. Nemotron-service produce suggerimento o fallback.
 16. Frontend mostra card animata.
 17. TTS legge suggerimento.
-

19. Flusso dati: TomTom prediction

Scenario:

utente entra in nuova cella locale

Passaggi:

1. Frontend calcola cella locale da circa 250 m.
2. Se cache frontend valida, non chiama backend.
3. Se serve, chiama:

```
POST /ingestion/traffic/tomtom/publish
```

con:

```
{
  "lat": "...",
  "lon": "...",
```

```
"radius_m": 500
}
```

4. Ingestion-service controlla Redis cache.
5. Se cache backend valida, quota TomTom consumata 0.
6. Se non valida, controlla budget.
7. Chiama TomTom Flow sui punti campione.
8. Chiama TomTom Incidents sulla bbox.
9. Converte dati in `traffic_pressure` ed `event_pressure`.
10. Pubblica evento su RabbitMQ.
11. Zone-service consumer riceve evento.
12. Zone-service associa segnale ai segmenti vicini.
13. Scrive `segment:signals:*` in Redis.
14. Frontend aspetta circa 700 ms.
15. Frontend ricarica heatmap/prediction.

Punto da dire:

TomTom non viene chiamato a ogni pressione o movimento. Viene protetto da cache frontend, cache backend, cella geografica e budget guard.

20. Flusso dati: report utente

Scenario:

utente segnala "ho trovato posto"

Passaggi:

1. Frontend invia report.
 2. Gateway inoltra a zone-service.
 3. Zone-service salva report.
 4. Zone-service applica rate limit/sessione.
 5. Pubblica evento su RabbitMQ.
 6. Prediction-service o zone-service considerano report recenti.
 7. Report positivo aumenta percentuale.
 8. Report negativo la diminuisce.
 9. Segmenti vicini possono essere influenzati con decadimento.
-

21. Flusso dati: suggerimento AI

Scenario:

utente passa da Via A a Via B e cambia percentuale

Passaggi:

1. Frontend rileva cambio segmento/percentuale.
2. Prepara contesto:
 - segmento corrente;
 - percentuale;
 - status;
 - segmenti vicini;
 - parcheggi vicini;
 - ID validi.
3. Chiama `/ai/explain`.
4. Nemotron-service controlla cache.
5. Se key Nemotron presente:
 - invia prompt;

- attende risposta;
 - valida JSON;
 - valida target IDs.
6. Se key assente:
 - usa fallback simulato.
 7. Frontend mostra card suggerimento.
 8. Evidenzia target suggeriti con anello verde rotante.
 9. Chiama `/ai/tts` se ElevenLabs disponibile.
 10. Se TTS termina, card si chiude con animazione.

22. Cloud AWS

Servizi usati

Locale	AWS
Docker Compose / k3d	EKS
container PostgreSQL/PostGIS	RDS PostgreSQL
container Redis	ElastiCache Redis
container RabbitMQ	Amazon MQ for RabbitMQ
immagini locali	ECR
env locale	SSM Parameter Store / Secret Kubernetes
log container	CloudWatch Logs
script down locale	Lambda + EventBridge + GitHub Actions

Terraform

Cartella:

`infrastructure/terraform/aws`

Risorse principali:

- ECR repositories per ogni servizio;
- lifecycle policy ECR;
- VPC;
- subnet pubbliche/private;
- NAT Gateway;
- EKS cluster;
- EKS managed node group ARM64 `t4g.small`;
- RDS PostgreSQL 16;
- ElastiCache Redis 7;
- Amazon MQ RabbitMQ 3.13;
- CloudWatch log groups;
- Lambda auto-down;
- IAM roles/policies;
- EventBridge Scheduler;
- backend remoto S3 per state.

Perche' `enable_cloud_stack=false` di default

Per evitare costi accidentali.

Con default false:

- si possono creare risorse piu' leggere come ECR;
- non si accendono EKS/RDS/Redis/MQ.

Per demo reale:


```
enable_cloud_stack=true
```

Node group

Configurazione:

```
instance_types = ["t4g.small"]
ami_type = "AL2023_ARM_64_STANDARD"
min_size = 1
desired_size = 2
max_size = 2
```

Motivo:

- ARM64 costa meno;
- sufficiente per demo;
- 2 nodi danno dimostrazione piu' realistica.

RDS

Configurazione:

```
engine = postgres
version = 16
class = db.t4g.micro
storage = 20 GB
publicly_accessible = false
skip_final_snapshot = true
backup_retention = 1
```

Nota:

- `skip_final_snapshot=true` e' scelto per demo breve;
- in produzione non sarebbe consigliato.

Redis AWS

Configurazione:

```
engine = redis
node_type = cache.t4g.micro
num_cache_nodes = 1
```

Amazon MQ

Configurazione:

```
engine_type = RabbitMQ
engine_version = 3.13
host_instance_type = mq.m5.large
publicly_accessible = false
```

Nota:

- Amazon MQ e' costoso rispetto agli altri servizi;
- scelto per continuita' con RabbitMQ locale.

Auto-spegnimento

Componenti:

- Lambda `auto_down_dispatcher`;
- EventBridge Scheduler;
- token GitHub letto da SSM;

- workflow GitHub cloud-down.yml.

Idea:

Dopo 4 ore EventBridge invoca Lambda.
Lambda chiama GitHub Actions.
GitHub Actions esegue cloud-down.
Terraform spegne risorse costose.

Da dire:

L'auto-down riduce il rischio, ma non sostituisce il controllo manuale. Dopo la demo si verifica sempre con `scripts/cloud_status.sh`.

23. Kubernetes

Manifest:

infrastructure/k8s/local-demo.yaml
infrastructure/k8s/cloud-demo.yaml

Kubernetes locale

Usa:

- k3d;
- cluster k3s in Docker;
- immagini buildate localmente;
- import immagini nel cluster;
- pod per PostgreSQL, Redis, RabbitMQ;
- microservizi come deployment;
- port-forward.

Script:

scripts/k3d_prepare_local.sh
scripts/k8s_apply_local.sh
scripts/k8s_import_osm_local.sh
scripts/k8s_smoke_test.sh
scripts/cloud_sim_local_up.sh

Kubernetes cloud

Usa:

- EKS;
- immagini ECR;
- RDS/ElastiCache/Amazon MQ;
- ConfigMap/Secret generati da output Terraform e SSM;
- Load Balancer.

Script:

scripts/k8s_cloud_config_from_aws.sh
scripts/k8s_import_osm_cloud.sh
scripts/k8s_cloud_dry_run.sh

24. GitHub Actions

Workflow presenti:

```
.github/workflows/ci.yml
.github/workflows/docker-build.yml
.github/workflows/ecr-build.yml
.github/workflows/deploy-local.yml
.github/workflows/deploy-eks.yml
.github/workflows/terraform-plan.yml
.github/workflows/terraform-apply.yml
.github/workflows/cloud-down.yml
```

CI

Controlla:

- static checks;
- contratti;
- frontend checks;
- smoke;
- build.

ECR build

Costruisce immagini e le pubblica su ECR.

Richiede repository secrets:

- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY
- eventuale region/ECR base.

Terraform workflows

terraform-plan puo' funzionare anche senza credenziali con init senza backend in certi casi.

terraform-apply richiede credenziali.

Deploy EKS

Aggiorna immagini dei deployment su EKS usando ECR.

Cloud down

Workflow usato anche da Lambda auto-down.

25. Sicurezza e segreti

Principi:

- nessuna API key nel frontend;
- nessuna API key committata;
- .env locale opzionale;
- SSM in AWS;
- Kubernetes Secret derivato da SSM/output Terraform;
- fallback quando mancano segreti.

Locale

Variabili opzionali:

```
TOMTOM_API_KEY
NEMOTRON_API_KEY
ELEVENLABS_API_KEY
```

Se assenti:

- TomTom non viene chiamato;
- Nemotron usa fallback;
- ElevenLabs non genera audio;
- frontend usa TTS browser.

AWS

Script:

```
scripts/aws_ssm_sync_env.sh
scripts/aws_ssm_check_config.sh
scripts/k8s_secret_from_ssm.sh
```

SSM evita di mettere chiavi nei manifest.

26. Testing

Script principali:

```
scripts/static_checks.sh
scripts/smoke_test.sh
scripts/e2e_demo_test.sh
scripts/check_frontend.py
scripts/check_tomtom.py
scripts/check_osm_import.py
scripts/check_road_backed_segments.py
scripts/check_parking_overrides.py
scripts/check_scoring.py
scripts/check_contracts.py
scripts/k8s_smoke_test.sh
scripts/load_test.sh
```

Cosa testano

- che il frontend usi MapLibre;
- che non siano rimasti riferimenti Leaflet obbligatori;
- che gli endpoint principali rispondano;
- che OSM sia importato;
- che segmenti road-backed esistano;
- che prediction e scoring siano coerenti;
- che TomTom abbia guardrail;
- che Docker/Kubernetes deploy siano smoke-testabili.

Smoke test

Lo smoke test e' importante per l'esame perche':

- dimostra che non e' solo codice statico;
 - avvia path reali;
 - verifica gateway e servizi;
 - controlla risposte API.
-

27. Differenza tra locale, k3d e AWS

Locale Docker Compose

Vantaggi:

- facile;
- veloce;
- un comando;
- ideale per sviluppo.

Svantaggi:

- non dimostra orchestrazione reale;
- meno simile al cloud.

k3d

Vantaggi:

- simula Kubernetes;
- gratuito;
- utile didatticamente.

Svantaggi:

- usa comunque risorse PC;
- non usa servizi gestiti reali.

AWS

Vantaggi:

- architettura cloud reale;
- servizi gestiti;
- EKS/RDS/Redis/MQ reali;
- dimostra IaC.

Svantaggi:

- costa;
- richiede credenziali;
- richiede spegnimento;
- provisioning piu' lento.

28. Scelte progettuali principali

Perche' niente React

Risposta breve:

Per questa demo la complessita' era nella mappa, nei dati e nel cloud, non nello stato UI tipico di una SPA. HTML/CSS/JS vanilla e' sufficiente e riduce dipendenze.

Dettaglio:

- meno build tooling;
- meno problemi CI;
- meno peso frontend;
- piu' facile da servire con Nginx;
- coerente con richiesta successiva "nessun framework".

Perche' niente Spring Boot

Risposta:

Il backend e' stato diviso in microservizi FastAPI e il gateway e' Nginx. Spring Boot avrebbe aggiunto un secondo stack backend senza vantaggi concreti nella demo.

Perche' Nginx gateway

Motivi:

- semplice;
- standard;
- adatto come reverse proxy;
- basso overhead;
- facile in Docker/Kubernetes;
- gestisce timeout AI lunghi.

Perche' niente MongoDB

Risposta:

MongoDB era opzionale. I dati strutturati/geografici stanno meglio in PostGIS; lo stato temporaneo sta in Redis; gli eventi passano da RabbitMQ. Aggiungere MongoDB avrebbe aumentato complessita' senza beneficio nella demo.

Perche' RabbitMQ invece SQS/SNS

Risposta:

RabbitMQ funziona bene in locale e ha equivalente gestito Amazon MQ, quindi consente continuita' tra Docker Compose e AWS. SQS/SNS sarebbero alternative valide ma avrebbero cambiato il modello eventi in cloud.

Perche' EKS invece EC2 semplice

Risposta:

Il requisito chiedeva Kubernetes/cloud-native. EKS mostra orchestrazione gestita, deployment, service discovery e scalabilita'. EC2 semplice sarebbe costato meno ma meno coerente col requisito.

Perche' Terraform

Risposta:

Terraform rende l'infrastruttura riproducibile, revisionabile e distruttibile. Invece di cliccare nella console AWS, descriviamo risorse e dipendenze in codice.

Perche' Ansible solo parziale

Risposta:

Ansible e' stato usato per playbook didattici di setup locale/k3d/Multipass. Il provisioning cloud vero e' piu' adatto a Terraform, mentre il deploy applicativo e' gestito da script/Kubernetes.

29. Cosa succede senza API key

Questa e' una domanda probabile.

Senza API key:

- mappa funziona;
- segmenti OSM reali funzionano;
- heatmap funziona;
- prediction simulata/rule-based funziona;
- suggerimenti simulati funzionano;
- TTS browser funziona;
- TomTom live non viene chiamato;
- ElevenLabs non consuma crediti;

- Nemotron non consuma crediti.

Quindi il progetto e' riproducibile anche da chi clona la repo.

30. Cosa succede con API key

Con TomTom:

- ingestion-service puo' chiamare Flow/Incidents/Search;
- aggiorna segnali Redis;
- prediction usa segnali live;
- marker parcheggi possono arrivare da Search.

Con Nemotron:

- suggerimenti generati live;
- target segmenti/POI validati.

Con ElevenLabs:

- TTS audio naturale;
 - modello flash 2.5 per minor costo.
-

31. Limiti attuali

Da non nascondere all'esame.

Limiti:

- non e' un prodotto pronto per utenti reali;
- non ha storico reale di occupazione;
- prediction rule-based, non ML addestrato;
- TomTom non fornisce disponibilita' on-street reale;
- override strisce blu possono essere incompleti;
- routing reale non implementato;
- Android nativo non implementato;
- MongoDB non implementato;
- Spring Boot non implementato;
- cloud AWS ha costi e va spento.

Risposta matura:

Il progetto dimostra architettura, integrazione dati e UX predittiva. La precisione reale richiederebbe dataset storico o sensori/partner per occupazione parcheggi.

32. Possibili sviluppi futuri

1. Routing reale con OSRM/Valhalla/TomTom Routing.
2. Storico report utenti.
3. Modello ML supervisionato.
4. Migliore dataset ufficiale strisce blu.
5. App Android nativa.
6. Autenticazione utenti.
7. Dashboard osservabilita' Prometheus/Grafana.
8. IAM least privilege su AWS.
9. SQS/SNS come alternativa cloud-native.
10. Test visuali automatici della GUI.
11. Multi-citta'.

12. Pipeline OSM automatica.
 13. Aggiornamento periodico override comunali.
-

33. Domande probabili e risposte pronte

Perche' avete usato MapLibre?

Per avere una mappa WebGL con pitch, bearing, heatmap e layer vettoriali. Leaflet sarebbe stato piu' semplice ma meno adatto a un'esperienza da navigatore.

Perche' i segmenti e non le zone?

Le zone sono troppo grandi. L'utente deve sapere quale via provare. I segmenti stradali permettono consigli locali e visivamente coerenti con la strada.

Come calcolate la percentuale?

Partiamo da una baseline per tipo sosta, poi applichiamo correzioni da report, segnali Redis/TomTom, ora/giorno e distanza. Il risultato viene limitato e convertito in status visuale.

La percentuale e' certa?

No. E' una stima. La confidence indica quanto siamo sicuri. La segnaletica reale va sempre verificata.

TomTom dice se ci sono posti liberi?

No, con le API attuali no. TomTom fornisce traffico, incidenti e POI parcheggi. La disponibilita' on-street e' stimata dal nostro backend.

Come evitate di consumare troppe chiamate TomTom?

Cache frontend, cache backend Redis, celle da circa 250 m, TTL 5 minuti, raggio 500 m, budget guard al 5% quota mensile.

Perche' usate Redis?

Per dati temporanei veloci: sessioni live, segnali, cache, budget, eventi recenti.

Perche' usate RabbitMQ?

Per disaccoppiare eventi asincroni: report, traffico, scenari demo. Inoltre Amazon MQ permette una migrazione cloud coerente.

Perche' PostgreSQL/PostGIS?

Perche' il dominio e' geospaziale: distanze, coordinate, nearest segment, bbox, geometrie stradali.

Cosa fa il gateway?

Nginx riceve le richieste esterne e le instrada al microservizio corretto. Nasconde la topologia interna e centralizza CORS/timeouts.

Perche' non avete usato Spring Boot?

Per evitare doppio stack. FastAPI era gia' usato per dati/predizione/AI; Nginx risolve bene il gateway.

Come funziona l'AI?

Il backend prepara contesto strutturato, Nemotron genera un JSON con consiglio e target, il backend valida gli ID, il frontend mostra e legge il suggerimento.

Se Nemotron non risponde?

Il servizio usa fallback simulato o non blocca la GUI. La prediction continua comunque.

Se ElevenLabs manca?

Il frontend usa TTS browser o resta senza voce naturale. Non blocca il sistema.

Che differenza c'è tra Docker Compose e Kubernetes locale?

Compose avvia container direttamente. k3d avvia un cluster Kubernetes locale, con deployment/service simili al cloud.

Cosa fa Terraform?

Crea e gestisce infrastruttura AWS: ECR, VPC, EKS, RDS, Redis, RabbitMQ, log, Lambda auto-down.

Come spegnete AWS?

Con `CONFIRM_DESTROY=destroy-parcheggia-dev scripts/cloud_down.sh`. Inoltre esiste auto-down via Lambda/EventBridge che chiama GitHub Actions.

Perche' auto-down?

Per ridurre rischio costi. EKS/RDS/ElastiCache/Amazon MQ costano finche' accesi.

Perche' non MongoDB?

Non serviva: PostGIS copre dati geografici persistenti, Redis stato veloce, RabbitMQ eventi. MongoDB avrebbe aggiunto complessita'.

Come fate a far funzionare il progetto a chi clona la repo?

Docker Compose importa automaticamente dati OSM e override. Senza API key usa fallback simulati.

Qual e' il punto piu' cloud del progetto?

La stessa architettura gira in locale, in Kubernetes locale e su AWS con servizi gestiti equivalenti. Terraform rende riproducibile il cloud.

34. Comandi utili da ricordare

Locale

```
docker compose up -d --build
docker compose ps
scripts/smoke_test.sh
scripts/e2e_demo_test.sh
docker compose down
docker compose down -v
```

Kubernetes locale

```
scripts/cloud_sim_local_up.sh
scripts/cloud_sim_local_down.sh
```

AWS

```
aws configure --profile parcheggia-dev
aws sts get-caller-identity --profile parcheggia-dev
scripts/cloud_plan.sh
CONFIRM_APPLY=apply-parcheggia-dev scripts/cloud_demo_up.sh
scripts/cloud_status.sh
CONFIRM_DESTROY=destroy-parcheggia-dev scripts/cloud_down.sh
```

Debug

```
docker compose logs -f api-gateway
docker compose logs -f zone-service
docker compose logs -f prediction-service
docker compose logs -f ingestion-service
docker compose logs -f nemotron-service
```

RabbitMQ:

```
http://localhost:15672
parcheggia / parcheggia
```

Docs API locali:

```
http://localhost:8001/docs zone-service
http://localhost:8002/docs ingestion-service
http://localhost:8003/docs nemotron-service
http://localhost:8004/docs prediction-service
http://localhost:8005/docs location-service
http://localhost:8006/docs admin-service
```

35. File piu' importanti da citare

Frontend:

```
frontend/app.js
frontend/styles.css
frontend/index.html
frontend/assets/catania.pmtiles
```

Gateway:

```
services/api-gateway/nginx.conf
```

Servizi:

```
services/zone-service/app/main.py
services/prediction-service/app/main.py
services/prediction-service/app/scoring.py
services/location-service/app/main.py
services/ingestion-service/app/main.py
services/ingestion-service/app/tomtom.py
services/nemotron-service/app/main.py
services/admin-service/app/main.py
```

Dati:

```
services/zone-service/migrations/001_create_zones.sql
data/osm/catania_segments.sql
data/osm/catania_blue_overrides.sql
data/parking_overrides/catania_blue_zones.csv
data/synthetic/demo_scenarios.json
```

Cloud:

```
infrastructure/k8s/local-demo.yaml
infrastructure/k8s/cloud-demo.yaml
infrastructure/terraform/aws/main.tf
infrastructure/terraform/aws/variables.tf
infrastructure/terraform/aws/backend.tf
infrastructure/terraform/aws/lambda/auto_down_dispatcher.py
```

Script:

```
scripts/cloud_demo_up.sh
scripts/cloud_down.sh
scripts/cloud_sim_local_up.sh
scripts/k3d_prepare_local.sh
scripts/aws_ecr_push.sh
scripts/aws_ssm_sync_env.sh
scripts/static_checks.sh
scripts/smoke_test.sh
```

CI:

```
.github/workflows/ci.yml
.github/workflows/ecr-build.yml
.github/workflows/deploy-eks.yml
.github/workflows/cloud-down.yml
```

36. Frase finale efficace per l'esame

Una buona sintesi da dire:

ParcheggIA non e' solo una mappa dei parcheggi. E' una architettura cloud-native end-to-end: importa dati OSM in PostGIS, calcola stime segment-level, aggiorna segnali tramite Redis e RabbitMQ, integra provider esterni in modo parsimonioso e sicuro, mostra tutto in una GUI MapLibre stile navigatore e puo' girare sia in locale con Docker Compose, sia su Kubernetes locale, sia su AWS tramite Terraform. La demo resta funzionante anche senza API key grazie ai fallback.

37. Cosa dire se chiedono "cosa manca per produzione?"

Risposta consigliata:

Per produzione servirebbero dati reali storici di occupazione, routing reale, autenticazione, osservabilita' completa, policy IAM least privilege, pipeline CI/CD fully automated, monitoraggio costi, test visuali, gestione multi-citta' e un modello predittivo validato su dati reali. L'attuale progetto e' una demo cloud-native completa, non un servizio pubblico certificato.